



NIYAMLENS

EVIDENCE BEFORE VERDICT

SIH26034 · TEAM HANDBOOK

Feature & Verification Guide

A practical runbook for demonstrating, testing and defending the NiyamLens packaged-commodity inspection prototype.

31 FEATURES	54/54 TESTS PASS	3 OCR PATHS	YES OFFLINE OCR
-----------------------	----------------------------	-----------------------	---------------------------

Public prototype (0.3.0 redeploy pending): <https://niyamlens-sih26034.vercel.app>

Source: <https://github.com/DuvvuruDeepakReddy18/NiyamLens-SIH26034>

Release 0.3.0 · Verified 3 September 2026 · Source repository

Contents

Use this handbook as the team's shared source of truth. Every feature is paired with a reproducible proof and a clear claim boundary.

What the product does	4
Verification labels used in this guide	4
Ten-minute team smoke test	4
Feature map	5
Detailed features and how to verify them	5
1. Multi-panel evidence capture	5
2. Original-image integrity and derivative separation	6
3. Capture-quality gate	6
4. OCR-safe preprocessing	6
5. Four-corner perspective correction	7
6. Barcode detection and card-free flattening	7
7. Multilingual local OCR	7
7A. Deep scan and calibrated retake guidance	8
7B. Explicit connected OCR	8
8. Structured declaration extraction and visual grounding	8
9. Human correction without destroying original evidence	8
10. Package profiles and mandatory declarations	9
11. Flat and cylindrical panel-area calculation	9
12. Rule 7 / Table-I typography tiers	9
13. Per-panel physical calibration	10
14. Reference-card detection and depth capability check	10
15. Calibrated uncertainty and safe abstention	10
16. Deterministic rules-as-code	11
17. Rule 26 exemptions and carve-outs	11
18. Four explicit result states	11
19. Evidence finalization, local persistence and history	11
20. Evidence packet, JSON and Print/PDF	12
21. Hash-linked audit timeline and tamper detection	12
22. Officer/supervisor separation and reason-required review	12
23. Local assignment queue	12
24. Encrypted offline case transfer	13
25. Blind challenge mode	13
26. Validation Lab	13
27. Offline PWA and local OCR assets	14
28. Responsive/mobile interface	14

29. Transparent legal approval register	14
Automated verification commands	14
Verify pure logic	14
Verify production compilation	15
Verify the complete live workflow	15
Verify local OCR and grounding	15
Verify explicit connected OCR and safe failure	15
Compare standard and deep OCR on real photos	15
Verify fully offline operation	15
Demonstration kit to carry	16
What the team must not overclaim	16
Recommended seven-minute role split	16
Release sign-off checklist	17
Related documents	17

- **Problem statement:** SIH26034 — packaged-commodity declaration compliance
- **Release:** NiyamLens 0.3.0, local release candidate
- **Public application:** <https://niyamlens-sih26034.vercel.app> (redploy 0.3.0 before using the new OCR controls)
- **Source repository:** <https://github.com/DuvvuruDeepakReddy18/NiyamLens-SIH26034>
- **Last verified locally:** 3 September 2026

What the product does

NiyamLens turns photographs of a packaged commodity into a reviewable inspection packet. It preserves the original images, checks capture quality, performs multilingual browser OCR, extracts declarations, applies deterministic rules, handles measurement uncertainty and exemptions, records a hash-linked audit trail, and produces a printable evidence report.

The product is decision support for an authorised officer. It does not claim to issue a statutory determination.

Verification labels used in this guide

- **Automated:** exercised against the current local production build by browser automation.
- **Live automated:** repeated against the public Vercel deployment after that release is deployed.
- **Unit tested:** its underlying logic has a passing regression test.
- **Manual:** implemented and available, but requires a person, suitable physical packet, device capability or visual judgment.
- **Approval required:** implemented as prototype logic but cannot be presented as department-approved or enforcement-grade.

Ten-minute team smoke test

Use this before every presentation.

- 1 Open `http://127.0.0.1:5173` in current Chrome/Edge, or use the public URL after release 0.3.0 is deployed.
- 2 Go to **New inspection**.
- 3 Expand **Controlled test packets** and select **Compliant packet**.
- 4 Confirm the right-side assessment says **PASS**.
- 5 Select **Violation packet** and confirm the assessment says **FLAG** with seven flagged checks in the verified release.
- 6 Select **Rule 26 exemption packet** and confirm the assessment says **EXEMPT**.
- 7 Select **Evidence report**, then **Finalize inspection**.
- 8 Open **Inspection history**, reopen the saved record, and confirm **Audit chain verified**.
- 9 Open **Officer operations**, switch to **Supervising Officer**, enter a review reason and seal a disposition. Reopen the evidence and confirm the automated result is still displayed.
- 10 Open **Blind challenge**, start it and confirm **Controlled test packets** is no longer available.

If those ten checks pass, the core presentation path is ready. The controlled packets prove repeatability; the blind challenge with a judge-selected package proves the system is not tuned only to canned data.

Feature map

Area	Implemented capability	Best proof
Intake	Camera/file capture for up to four guided package-panel roles	Upload and assign front, MRP/date, entity/care and quantity/barcode views
Integrity	SHA-256 digest of every original image	Compare the hash before and after rotate/flatten; it remains unchanged
Image quality	Sharpness, contrast, brightness and glare checks	Upload a clear image, then an intentionally blurred/glared image
Image correction	Rotation, grayscale, contrast and four-corner perspective flattening	Flatten a skewed panel in TL → TR → BR → BL order
Barcode	Native GTIN reading and barcode-plane flattening	Scan a real EAN/UPC package in supported Chrome/Edge
OCR	Three-pass local OCR plus seven-pass tiled deep scan in supported language combinations	Disconnect after caching and compare standard with Deep scan small text
Connected OCR	Explicit opt-in Google Vision serverless route with preserved local fallback	Click only after consent; simulate an outage and confirm the transcript remains
Grounding	Clickable OCR declaration regions	Click an extracted MRP or quantity card and see its image box highlight
Extraction	MRP, quantity, date, entity, care, origin, unit price, FSSAI text and barcode parsing	Try punctuated M.R.P., a 14-digit FSSAI licence and an invalid GTIN check digit
Geometry	Flat and cylindrical principal-display-panel calculators	Enter dimensions and observe the calculated square-centimetre area
Typography	Calibrated font height and width-to-height checks	Measure a reference, glyph height and glyph width on the same panel
Uncertainty	Table-I boundary and physical-measurement abstention	Set area near 100 cm ² with ±5% uncertainty and observe REVIEW
Rules	Versioned deterministic Rule 6, Rule 7 and Rule 26 evaluation	Open each finding to see rule, reason and evidence
Exemptions	Small pack, tobacco, pan masala, fast-food, drug and medical-device profiles	Compare an 8 ml standard package with the same quantity marked pan masala
Challenge	Sealed unseen-package timer with canned fixtures disabled	Start Blind challenge in front of the judge
Evidence	IndexedDB register, history, JSON and printable/PDF report	Finalize, reload, reopen the record, export JSON and print
Audit	Hash-linked inspection and review events	Report must say Audit chain verified
Operations	Officer/supervisor separation, assignments and reason-required override	Officer cannot override; supervisor can only seal with a reason
Transfer	PBKDF2 + AES-256-GCM encrypted case bundles	Export with a passphrase; wrong-passphrase import must fail
Validation	JSON/CSV labelled-dataset metrics and failure analysis	Import labels and inspect accuracy, field metrics, false violations and abstention
Field resilience	Installable offline PWA with bundled OCR assets	Put the browser fully offline, reload and run OCR

Detailed features and how to verify them

1. Multi-panel evidence capture

What it does: Accepts camera capture or files for as many as four declaration panels. Front, back and side panels remain separate evidence planes and receive editable evidence roles.

Why it matters: Mandatory declarations are often split across a package. A single front photograph is not enough to support a missing-declaration finding.

Manual verification:

- 1 Open **New inspection**.
- 2 Select **Capture / upload package**.
- 3 Add one image, then use **Add package panel (1/4)** to add more.
- 4 Confirm a thumbnail appears for every panel, selecting it changes the active image, and **Active panel purpose** can be corrected.
- 5 Add four panels and confirm the upload control is disabled at the limit.

Expected proof: The page states the number of captured panels, and the evidence report displays every retained panel separately.

Status: Live automated for real file upload; mobile camera selection is manual/device-dependent.

2. Original-image integrity and derivative separation

What it does: Calculates SHA-256 before preprocessing. Rotation, grayscale, contrast and perspective correction create analysis derivatives without replacing the original digest.

Why it matters: An inspector can improve OCR readability without losing the chain back to the originally captured file.

Manual verification:

- 1 Upload a real image and note the first characters shown beside **SHA-256**.
- 2 Select **Rotate**, change **OCR contrast**, or use **Flatten panel**.
- 3 Confirm the displayed SHA-256 prefix remains identical.
- 4 Finalize and open the report; confirm the original digest is retained with the panel.

Automated proof: `npm run qa:ui reports originalHashPreserved: true.`

Status: Live automated and unit-backed cryptographic implementation.

3. Capture-quality gate

What it does: Scores sharpness, contrast, brightness and glare and gives recapture guidance.

Why it matters: Poor input should not silently become a legal accusation.

Manual verification:

- 1 Upload a well-lit, sharply focused label.
- 2 Record the **Capture quality _/100** panel and the sharpness, contrast and glare readings.
- 3 Take a deliberately blurred photograph and another with strong flash glare.
- 4 Upload each and confirm the score or warning changes and names the first detected issue.

Expected proof: Clear images say they are suitable for OCR; weak images display a warning instead of pretending the evidence is reliable.

Status: Implemented; thresholds require field-dataset calibration before enforcement use.

4. OCR-safe preprocessing

What it does: Provides 90-degree rotation, grayscale and 80–180% OCR contrast controls.

Manual verification:

- 1 Upload a sideways label and select **Rotate** until it is upright.
- 2 Toggle **OCR grayscale** and move the contrast slider.
- 3 Run OCR before and after on a difficult label and compare the confidence/transcript.
- 4 Confirm any previous physical calibration for that transformed panel is invalidated and must be measured again.

Expected proof: The analysis view changes; the original hash does not; stale pixel calibration is not silently reused.

Status: Live implementation; OCR improvement varies by package.

5. Four-corner perspective correction

What it does: Applies projective homography to flatten a skewed label panel.

Manual verification:

- 1 Upload an oblique photograph of a flat package.
- 2 Select **Flatten panel**.
- 3 Click top-left, top-right, bottom-right and bottom-left, in that order.
- 4 Confirm the prompt counts all four points and the image becomes front-facing.
- 5 Rerun OCR and recalibrate the transformed panel.

Expected proof: The UI reports that perspective was flattened, the original digest is preserved, and calibration must be recreated for the new pixel plane.

Status: Live automated via `perspectiveRectification: true`.

6. Barcode detection and card-free flattening

What it does: Uses Chromium's native `BarcodeDetector` for EAN-13, EAN-8, UPC-A, UPC-E, Code 128 and QR. When four barcode corners are available, the barcode plane can guide perspective correction.

Manual verification:

- 1 Use current Chrome/Edge and upload a clear barcode panel.
- 2 Select **Read barcode**.
- 3 Confirm the decoded value populates or can be copied into **Barcode / GTIN**.
- 4 If **Flatten from barcode** appears, select it and confirm the panel is corrected.
- 5 In a browser without the API, confirm the UI gives a support warning and permits manual 8–14 digit GTIN entry.

Critical explanation: Barcode geometry can flatten perspective; it does not prove absolute millimetres because barcode magnification varies. A physical reference is still required for font-size decisions.

Status: Implemented; device/browser and barcode-quality dependent.

7. Multilingual local OCR

What it does: Runs bundled Tesseract OCR in these selectable modes:

- English
- English + Hindi
- English + Telugu
- English + Tamil

Manual verification:

- 1 Upload an appropriate label.
- 2 Select the OCR language combination.
- 3 Select **Run browser OCR**.
- 4 Observe progress, the editable transcript and the separate reliability/engine-confidence values.
- 5 Click extracted declaration cards to inspect their image grounding.

Verified sample result: The bundled sample completes at 91% calibrated reliability and 92% engine confidence with ten parsed signals and ten mapped evidence regions. It makes zero requests to external OCR services.

Do not claim: Neither 91% reliability nor 92% engine confidence is field accuracy.

Status: Live automated by `npm run qa:ocr` and `npm run qa:pwa`.

7A. Deep scan and calibrated retake guidance

What it does: Adds four overlapping 2×2 detail tiles to the three whole-panel OCR passes, then scores reliability from pass agreement, engine confidence, capture quality and evidence volume. Low reliability produces a retake/deep-scan recommendation instead of a confident-looking verdict.

Manual verification:

- 1 Upload a difficult small-text panel.
- 2 Run normal browser OCR and preserve the transcript.
- 3 Run **Deep scan small text** and compare recovered declarations.
- 4 Confirm poor captures remain visibly low-reliability rather than inheriting a single optimistic engine score.

Measured pilot: Standard local OCR recovered 61.1% of pre-labelled tokens and deep scan recovered 67.5% on the same 17 untouched declaration-panel photos from ten products. The larger stress set includes glare, soft focus, rotation and curved packaging; these remain known failure modes. This pilot is too small for a field-accuracy claim.

Status: Automated by `npm run qa:ocr:real` and `npm run qa:ocr:real:deep`; reports are committed under `reports/`.

7B. Explicit connected OCR

What it does: Sends processed panels to Google Vision only after the inspector clicks **Connected OCR**. The API key stays server-side, the response is normalized into NiyamLens evidence, the action is audited, and provider failure leaves the previous transcript unchanged.

Manual verification:

- 1 Configure `GOOGLE_CLOUD_VISION_API_KEY` on a Vercel preview or production deployment.
- 2 Load a package and confirm browser OCR works without using the provider.
- 3 Explain the transfer and select **Connected OCR** only with consent.
- 4 Verify the returned transcript against the package; do not treat provider confidence as accuracy.
- 5 Remove/disable the key and confirm the UI reports unavailability while local OCR remains usable.

Status: Server API and browser contract tested by `npm test` and `npm run qa:ocr:connected`. A real provider call requires the team's own secret and billing-enabled project.

8. Structured declaration extraction and visual grounding

What it does: Converts the OCR transcript into candidate fields for product name, MRP, net quantity, pack/manufacture date, responsible entity, consumer-care details, country of origin, unit price, FSSAI licence text and barcode. It tolerates punctuated M.R.P. and validates GTIN check digits. OCR word boxes are mapped back to image regions.

Manual verification:

- 1 Run OCR on the sample or a clear real packet.
- 2 Inspect the **Structured evidence** grid.
- 3 Select **MRP**, **Net quantity** or another detected card.
- 4 Confirm the matching rectangle highlights on the correct captured panel.
- 5 Correct a transcript error in the text editor and select **Apply detected context**.

Expected proof: Every detected field shows its value, parser confidence, optional region confidence and a clickable evidence location.

Status: Live automated; extraction edge cases are unit tested.

9. Human correction without destroying original evidence

What it does: Lets the officer correct OCR text and add an optional interpretation note while keeping captured images, original OCR evidence and decision logic separate.

Manual verification:

- 1 Change one obvious OCR character in the transcript.
- 2 Add an **Officer interpretation** note.
- 3 Apply the detected context and open the evidence report.
- 4 Confirm the report preserves the evidence text and separately records the inspection context.

Critical explanation: This is not automatic translation. It is a human clarification field and never replaces the source image.

Status: Implemented; officer identity is local prototype identity.

10. Package profiles and mandatory declarations

What it does: Changes required checks according to the selected context:

- General retail package
- Food product
- Imported package
- Medical device
- Optional time-sensitive commodity

Manual verification:

- 1 Load **Compliant packet** and observe **PASS**.
- 2 Change the profile to **Imported package** without adding country of origin.
- 3 Confirm the new applicable check does not silently pass.
- 4 Enable **Time-sensitive commodity** without a best-before/use-by declaration and inspect the added finding.
- 5 Select **Medical device** and confirm the system defers applicable declaration/typography questions to specialist review instead of applying the ordinary profile as final law.

Status: Deterministic rules; unit tested; approval required before official use.

11. Flat and cylindrical panel-area calculation

What it does: Calculates principal-display-panel area for a flat panel and estimates visible cylindrical label area.

Manual verification — flat:

- 1 Enter width 10 cm and height 8 cm.
- 2 Confirm **Principal display panel** becomes 80 cm².

Manual verification — cylinder:

- 1 Enter bottle diameter, label height and visible-circumference percentage.
- 2 Confirm area updates using $\pi \times \text{diameter} \times \text{height} \times \text{coverage}$.
- 3 Explain that the officer must still confirm which physical surface legally constitutes the principal display panel.

Status: Implemented; physical dimension acquisition and legal PDP selection remain human inputs.

12. Rule 7 / Table-I typography tiers

What it does: Selects minimum character height from panel area:

Principal display panel area	Printed label	Formed/moulded text
$A \leq 50 \text{ cm}^2$	1.0 mm	2.0 mm
$50 < A \leq 100 \text{ cm}^2$	1.5 mm	3.0 mm
$100 < A \leq 500 \text{ cm}^2$	2.5 mm	4.0 mm
$500 < A \leq 2500 \text{ cm}^2$	4.0 mm	6.0 mm
$A > 2500 \text{ cm}^2$	6.0 mm	6.0 mm

It also checks that measured character width is at least one-third of height, subject to stated narrow-character exceptions requiring human confirmation.

Manual verification:

- 1 Load **Compliant packet**. Its 80 cm² panel, 20 mm/120 px reference, 11 px glyph height and 4.2 px width should pass the encoded measurements.
- 2 Load **Violation packet**. Its 7 px height and 2 px width should generate typography flags.
- 3 Expand each rule row to show the rule, reason and measured evidence.

Status: Unit tested and demonstrated by controlled packets; laboratory measurement validation is pending.

13. Per-panel physical calibration

What it does: Keeps reference, glyph height and glyph width measurements scoped to one panel so a scale from one image cannot be reused on another.

Manual verification:

- 1 Upload two panels taken at visibly different distances.
- 2 Select panel 1, choose **Reference**, and click both ends of a known reference.
- 3 Choose **Glyph height** and **Glyph width** and measure the same character.
- 4 Switch to panel 2 and confirm its measurement readout is empty.
- 5 Set **Known reference length** to the physical reference length, normally 20 mm for the demo marker.

Expected proof: Every report states the number of calibrated panels; the application never claims that one panel's pixel-to-mm ratio applies to another.

Status: Unit tested by "never reuses one panel calibration for another evidence plane."

14. Reference-card detection and depth capability check

What it does: Attempts to detect the bundled green 20 mm reference marker and reports WebXR depth capability.

Manual verification:

- 1 Load a controlled packet containing the green **20 mm REF** marker.
- 2 Select **Detect 20 mm card** and inspect its confidence/result.
- 3 If detection is weak, use the manual two-click **Reference** tool.
- 4 Select **Depth capability** and confirm the browser reports supported or unavailable.

Do not claim: A positive WebXR capability result is not a validated depth measurement. Manual reference calibration remains the defensible competition path.

Status: Implemented; physical accuracy validation is pending.

15. Calibrated uncertainty and safe abstention

What it does: Propagates panel-area and measurement uncertainty. If an interval crosses a Table-I area boundary or font threshold, the result becomes **REVIEW/MANUAL REVIEW** rather than a guessed pass or violation.

Manual verification:

- 1 Use a non-exempt package and enter panel area 99 cm².
- 2 Set panel-area uncertainty to $\pm 5\%$.
- 3 Confirm the interval crosses the 100 cm² tier and the relevant finding requests review.
- 4 Lower OCR confidence or omit physical calibration and confirm missing evidence produces review rather than an automatic violation where the confidence policy applies.

Status: Unit tested at the 100 cm² boundary and low-OCR-confidence cases.

16. Deterministic rules-as-code

What it does: The application evaluates versioned JavaScript rule logic. No LLM decides PASS, FLAG, REVIEW or EXEMPT.

Manual verification:

- 1 Open **Rule library**.
- 2 Confirm rule pack LMPC-RC-2026.09-RC4 and matrix LMPC-MATRIX-2026.09-RC4 are visible.
- 3 Inspect the Rule 6, Rule 7 and Rule 26 cards, applicability matrix, edge cases, source links and external approval register.
- 4 Return to an inspection and expand a finding to see its authority, reason and evidence.

Expected proof: The same inputs always produce the same outcome, and the report records the rule-pack version.

Status: 44 automated tests pass, including legal boundaries and failure cases; departmental approval is still pending.

17. Rule 26 exemptions and carve-outs

What it does: Encodes these prototype profiles:

- Standard weight/measure package of 10 g or 10 ml or less → eligible small-package exemption.
- Tobacco/tobacco product → ordinary small-package exemption not applied.
- Pan masala → ordinary small-package exemption not applied.
- Restaurant/hotel fast food → selected Rule 26 profile.
- Specified drug formulation → specialist classification path.
- Medical device → specialist rules/review path.

Manual verification:

- 1 Select **Rule 26 exemption packet**: 8 ml, standard commodity. Expected overall status: **EXEMPT**.
- 2 Change only **Rule 26 commodity class** to **Tobacco / tobacco product**. Expected: not exempt.
- 3 Change it to **Pan masala (2025 carve-out)**. Expected: not exempt.
- 4 Enter 10.1 ml as standard. Expected: the ≤ 10 ml exemption no longer applies.

Automated proof: Edge tests cover exactly 10 g, 10.1 g, 8 ml standard, 8 g tobacco and 8 g pan masala.

Status: Unit tested; legal sign-off required before enforcement use.

18. Four explicit result states

What it does: Separates:

- **PASS:** supplied evidence passed all evaluated rules.
- **FLAG:** at least one violation remains outside uncertainty bounds.
- **MANUAL REVIEW:** evidence is incomplete, low-confidence or too close to a threshold.
- **EXEMPT:** selected package profile meets an encoded exemption.

Manual verification: Use **Compliant packet**, **Violation packet**, **Rule 26 exemption packet**, and the $99 \text{ cm}^2 \pm 5\%$ boundary example respectively.

Status: Live and unit tested.

19. Evidence finalization, local persistence and history

What it does: Saves sealed inspections in the browser's IndexedDB and shows metadata in dashboard/history views.

Manual verification:

- 1 Complete an inspection and select **Finalize inspection**.
- 2 Go to **Inspection history** and locate the generated record ID.
- 3 Reload the page and confirm the record remains.

- 4 Open **Command view** and confirm its counts/attention view reflect saved records.

Critical limitation: Data is tied to that browser profile. Clearing site data, using private browsing or changing devices will not show the same records unless an encrypted bundle is transferred.

Status: Live automated for finalize/dashboard/report reopen; no central server synchronization.

20. Evidence packet, JSON and Print/PDF

What it does: Produces a report containing outcome, evidence score, rule-by-rule findings, captured images, hashes, extracted fields, OCR confidence, geometry, calibration, rule-pack version, audit events and any supervisor disposition.

Manual verification:

- 1 Finalize an inspection and select **Evidence report**.
- 2 Confirm **Audit chain verified** and inspect every report section.
- 3 Select **Export JSON** and verify a record file downloads.
- 4 Select **Print / PDF** and save a PDF.
- 5 Reopen the record from history and confirm the same result.

Status: Live automated for report creation/reopening; file-save dialogs remain manual.

21. Hash-linked audit timeline and tamper detection

What it does: Every material action adds an event containing the previous hash. Verification recomputes the chain.

Manual verification:

- 1 Capture, transform, OCR and finalize an inspection.
- 2 Open the evidence report and inspect **Hash-linked local audit timeline**.
- 3 Confirm **Audit chain verified**, the event count and final hash prefix.

Automated proof: Unit tests verify an intact chain and intentionally altered chain detection.

Critical limitation: This detects local record mutation but is not an external timestamp, blockchain or WORM evidence store.

Status: Live and unit tested.

22. Officer/supervisor separation and reason-required review

What it does: Demonstrates different local roles. A supervisor disposition adds a new audit event while preserving the original automated status.

Manual verification:

- 1 Save an inspection and open **Officer operations**.
- 2 Choose **Field Officer 01**; confirm **Review / override** is disabled.
- 3 Choose **Supervising Officer**; open **Review / override**.
- 4 Leave the reason blank and confirm the seal action is disabled.
- 5 Select a disposition, provide the physical/legal reason and seal it.
- 6 Reopen **Evidence** and confirm **Automated status preserved as ...** appears beside the supervisor disposition.

Status: Live automated; production identity must use real SSO/RBAC.

23. Local assignment queue

What it does: Lets the supervisor create a local premises/package assignment.

Manual verification:

- 1 Open **Officer operations** as **Supervising Officer**.
- 2 Enter a premises/package reference and select **Assign**.

- 3 Confirm a generated ASN- . . . record appears as assigned.
- 4 Reload and confirm it remains in local storage.

Status: Implemented local workflow; not a department-wide dispatch system.

24. Encrypted offline case transfer

What it does: Exports/imports evidence bundles using PBKDF2-SHA256 with 600,000 iterations and AES-256-GCM. Large image-bearing records and legacy-v1 imports are supported.

Manual verification:

- 1 Open **Officer operations** with at least one sealed inspection.
- 2 Enter a passphrase of at least eight characters.
- 3 Select **Encrypt export** and confirm a `.niyamlens.enc.json` file downloads.
- 4 In a clean browser profile, enter the same passphrase and import the file. Confirm the case enters the local register.
- 5 Repeat with a wrong passphrase; import must fail and no record should be created.

Security practice: Transfer the passphrase through a different channel from the bundle.

Status: Unit tested for successful round-trip, wrong password, short password and large records.

25. Blind challenge mode

What it does: Starts a timestamped, coded unseen-package run and hides all controlled packets.

Manual verification:

- 1 Ask a judge to choose and retain a random package.
- 2 Open **Blind challenge** and select **Start blind challenge**.
- 3 Confirm the timer/ribbon appears and the page says controlled packets are disabled.
- 4 Continue the inspection using only the judge's package.
- 5 Finalize and show the challenge code in the evidence report.

Why this is the strongest feature: It makes the prototype falsifiable in the room and directly answers “was this tuned only for the demo image?”

Status: Live automated for timer start and controlled-fixture lockout.

26. Validation Lab

What it does: Imports labelled JSON or CSV and reports:

- Verdict accuracy
- Declaration-field precision, recall and F1
- Extracted-value accuracy when expected values are supplied
- False-violation rate
- Abstention/manual-review rate
- Failure examples

Manual verification:

- 1 Open **Validation lab**.
- 2 Confirm it says the loaded records are synthetic regression fixtures.
- 3 Download **Dataset template**.
- 4 Import a valid labelled JSON/CSV file.
- 5 Confirm the sample count and metrics change.
- 6 Import malformed data and confirm a readable import error appears.

Do not claim: Metrics from the built-in synthetic fixtures are not field accuracy. Field claims require a frozen, independently labelled real-package dataset.

Status: Parser and metrics are unit tested; real field dataset is pending.

27. Offline PWA and local OCR assets

What it does: Installs a service worker and caches the application shell plus English, Hindi, Telugu, Tamil, worker and WebAssembly OCR assets.

Manual verification:

- 1 Open the production URL online and allow the first cache installation to finish. The OCR bundle is roughly 28 MB.
- 2 In Chrome/Edge Developer Tools, open **Network** and select **Offline**.
- 3 Reload the page.
- 4 Upload the sample or a local label and run browser OCR.
- 5 Confirm the application and OCR both work without network access.

Automated proof: The production-preview check found service worker `niyamlens-shell-v7`, 16 cached shell/OCR resources, successful offline reload and successful offline OCR at 91% reliability / 92% engine confidence.

Status: Automated on the current production build; repeat on the public URL after deployment.

28. Responsive/mobile interface

What it does: Provides a collapsible navigation menu and camera-friendly workflow at phone viewport sizes.

Manual verification:

- 1 Open the live URL on an Android phone in Chrome.
- 2 Open navigation and move among every route.
- 3 Use **Capture / upload package** and grant camera access.
- 4 Confirm the image, form, verdict and report are readable without overlap.

Status: The 390 × 844 navigation flow has live automated coverage; real-device camera usability is manual.

29. Transparent legal approval register

What it does: Shows unresolved approvals instead of presenting prototype interpretation as official law. The pending gates are legal sign-off, laboratory measurement validation, field-dataset approval and security review.

Manual verification:

- 1 Open **Rule library**.
- 2 Find **NO FABRICATED APPROVAL — External approval register**.
- 3 Confirm every unresolved gate and its responsible owner are visible.

Status: Implemented. These gates are genuine remaining work, not cosmetic warnings.

Automated verification commands

Run these from the `niyamlens` repository after `npm install`.

Verify pure logic

```
npm test
```

Expected release baseline: 44 tests, 44 passed, 0 failed.

Verify production compilation

```
npm run build
```

Expected: Vite exits successfully and creates dist.

Verify the complete live workflow

```
$env:NIYAMLENS_BASE_URL='https://niyamlens-sih26034.vercel.app/'
$env:NIYAMLENS_BROWSER_PATH='C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe'
npm run qa:ui
```

Expected key results:

```
realUpload: true
perspectiveRectification: true
originalHashPreserved: true
structuredExtraction: true
flagCount: 7
reviewCount: 0
compliantDemo: true
controlledPacketOcr: true
supervisorOverride: true
supervisorReportReopened: true
blindChallenge: true
errors: []
```

Verify local OCR and grounding

```
$env:NIYAMLENS_BASE_URL='https://niyamlens-sih26034.vercel.app/'
npm run qa:ocr
```

Expected release baseline: 91% reliability / 92% engine confidence, product FIELD HARVEST, ten parsed signals, ten mapped regions, no external requests and no browser errors.

Verify explicit connected OCR and safe failure

```
npm run qa:ocr:connected
```

Expected results: explicit opt-in request, connected result visible, FSSAI text detected, simulated provider outage preserves evidence, and no unexpected browser errors.

Compare standard and deep OCR on real photos

```
npm run dataset:real:fetch
npm run qa:ocr:real
npm run qa:ocr:real:deep
```

Current pilot baseline: 61.1% standard versus 67.5% deep token recall across 17 declaration-panel photos from ten products; zero manual corrections. These are pilot OCR-recall measurements, not compliance accuracy.

Verify fully offline operation

```
$env:NIYAMLENS_BASE_URL='https://niyamlens-sih26034.vercel.app/'
npm run qa:pwa
```

Expected key results:

```
hasShellAssets: true
hasOcrAssets: true
offlineReload: true
offlineOcr: true
errors: []
```

If the test runner reports `spawn EPERM` on Windows, rerun the terminal with permission to create child processes. That error occurs before application assertions run and is not a failed rule/OCR assertion.

Demonstration kit to carry

- One clearly compliant retail package with declarations spread across multiple panels.
- One package with an obvious missing/weak declaration for explaining FLAG.
- One standard sachet at or below 10 g/10 ml.
- One tobacco or pan-masala example at or below 10 g/10 ml for the exemption-carve-out comparison.
- A flat 20 mm high-contrast reference marker and a ruler/vernier caliper.
- A glossy cylindrical bottle to demonstrate uncertainty and safe abstention.
- A laptop with current Chrome/Edge, charger and the PWA opened once online.
- An Android phone with the PWA cached, plus a power bank.
- Local copies of sample images and the encrypted case bundle.
- A hotspot, while keeping the offline route ready as the fallback.

What the team must not overclaim

Safe statement	Unsafe statement
"The bundled sample produced 91% reliability and 92% engine confidence."	"Our field OCR accuracy is 91%."
"Deep scan improved pilot token recall from 61.1% to 67.5% on our 17-photo stress set."	"Deep scan is 67.5% accurate on Indian labels."
"The prototype evaluates a versioned rules-as-code interpretation."	"The government has approved every encoded legal interpretation."
"Calibration and uncertainty support a reviewable measurement."	"Any phone photograph gives enforcement-grade millimetres."
"The local hash chain detects record mutation."	"This is blockchain/WORM evidence."
"Officer and supervisor separation is demonstrated locally."	"This already has production identity and departmental SSO."
"The application works offline after its assets are cached."	"All records synchronize across devices offline."
"The system supports officer decisions."	"The AI issues statutory violation notices."

Recommended seven-minute role split

- 1 **Presenter:** frames the regulatory problem and asks for a random package.
- 2 **Operator:** captures, corrects perspective, runs OCR and clicks grounded fields.
- 3 **Measurement lead:** explains panel area, calibration and uncertainty.
- 4 **Rules lead:** explains deterministic checks, Rule 26 and specialist deferral.
- 5 **Evidence/security lead:** opens audit report, supervisor review and encrypted export.
- 6 **Validation/deployment lead:** shows the Validation Lab, offline mode and answers architecture questions.

All six members should rehearse the ten-minute smoke test. The operator and presenter should separately rehearse the blind challenge using at least five unseen products.

Release sign-off checklist

- ☐ Public URL opens in Chrome/Edge.
- ☐ Controlled compliant packet returns PASS.
- ☐ Controlled violation packet returns FLAG.
- ☐ Controlled 8 ml standard packet returns EXEMPT.
- ☐ Pan masala/tobacco carve-out does not receive that exemption.
- ☐ Real sample OCR completes with grounded fields.
- ☐ Perspective correction preserves the original hash.
- ☐ Boundary uncertainty produces review.
- ☐ Finalized record survives reload.
- ☐ Report says Audit chain verified.
- ☐ Supervisor cannot seal without a reason.
- ☐ Automated status remains after supervisor disposition.
- ☐ Encrypted export imports with the correct passphrase and rejects a wrong one.
- ☐ Blind challenge hides controlled packets.
- ☐ Offline reload and OCR work.
- ☐ No one presents synthetic validation metrics as field accuracy.
- ☐ Legal, measurement, dataset and security approval gates are disclosed.

Related documents

- **Seven-minute judge demonstration**
- **Architecture and trust model**
- **Hybrid OCR architecture decision**
- **Legal Metrology field-dataset schema**
- **Field validation protocol**
- **Legal review register**
- **Deployment readiness**